

独自スクリプト言語からLuaへの変換ツール

1. プロジェクト基本情報

項目	内容
プロジェクト概要	業務プロジェクトで使用されていた独自スクリプト言語をLuaへ一括変換するツールの開発
OS	Windows 10
開発言語	Python
IDE	VS Code
バージョン管理	Perforce
使用ライブラリ	PLY (Python Lex-Yacc)
担当領域	言語仕様の調査、字句・構文解析、AST設計、意味変換、Luaコード生成

2. 開発背景・目的・担当範囲

2.1 開発背景

業務プロジェクトで使用されていた既存の独自スクリプトを、Lua環境へ移行する必要がありました。一方、参照できたのは既存スクリプトのみであり、変換元言語の処理系のソースコードや正式な言語仕様は提供されていませんでした。

また、大量のスクリプトを手作業で書き換える方法では、変換漏れや実行上の意味の変化が発生する可能性がありました。そのため、変換元言語の構造を解析し、一定の規則に基づいてLuaコードを生成する一括変換ツールが必要でした。

2.2 開発目的

- 変換元言語の構文と実行上の意味を可能な限り維持したLuaコードの生成
- 変換後のコードを後続システムへ組み込み、継続して保守できる状態の実現
- 手作業による大量変換に伴う変換漏れや意味の変化の抑制

2.3 個人の担当範囲

本ツールについて、以下の調査、設計、実装を担当しました。

- 既存スクリプトの調査と、変換元言語の字句・構文規則の整理
- LexerおよびParserの設計・実装
- AST（抽象構文木）の設計・実装
- 変換元言語とLuaの仕様差を補う意味変換の実装
- Luaコード生成、コメントおよび型注釈の出力処理の実装

3. 技術調査と方式選定

3.1 言語処理系に関する技術調査

開発着手時には言語処理系に関する技術資料と関連書籍を調査し、字句解析、構文解析、AST、コード生成からなる基本構成を学びました。その中で、変換元のソースコードをASTへ変換し、ASTを中間表現として解析やコード生成を行う方式が、コンパイラや言語変換ツールで一般的に利用されていることを確認しました。

3.2 ASTを採用した理由

変換元言語とLuaには、表面的な構文だけでなく、制御構造や演算子の意味にも違いがあります。このため、ソースコードに文字列置換を直接適用する方式ではなく、プログラムの構造をASTとして保持する方式が適切だと判断しました。

ASTを中間層とすることで、構文解析、意味変換、Luaコード生成の責務を分離しました。また、変換規則をASTノード単位で実装できるため、言語仕様の調査によって新たな記述パターンが判明した場合にも、規則を段階的に追加できる構成としました。

3.3 PLYを採用した理由

PLYを採用した理由は、LexerとParserを構築するための仕組みがあらかじめ提供されており、字句解析器と構文解析器の基盤を一から実装する必要がなかったためです。

本ツールでは、PLY上に変換元言語のToken規則と文法規則を定義し、各文法規則からASTノードを構築しました。これにより、解析器の基盤実装にかかる工数を抑え、変換元言語の仕様調査と変換処理の実装に注力しました。

4. 全体の処理構成

変換元スクリプト

↓

字句解析 (Lexer)

↓

構文解析 (Parser)

↓

AST構築

↓

型・スコープ・関数情報の解析

↓

言語間の仕様差を補う意味変換

↓

Luaコード生成

ASTには構文構造に加えて、型、変数の作用域、ノードの親子関係、構文要素の前後にあるコメントを保持させました。これらの情報を意味変換とLuaコード生成で利用する構成としました。

また、複数ファイルを事前解析して関数シグネチャを収集した後に、Luaへの変換を行う二段階構成を採用しました。これにより、単一ファイル内の情報だけでなく、複数ファイルにまたがる関数情報を変換処理で参照できるようにしました。

5. 代表的な技術課題と解決方法

5.1 正式な言語仕様がいない状態からの文法整理

課題

変換元言語の処理系の実装と正式な文法資料がなかったため、既存スクリプトから、変換に必要な言語仕様を整理する必要がありました。

調査と判断

既存スクリプトから、キーワード、型、演算子、文字列、コメント、関数、制御構文の使用例を収集しました。その上で、個別のスクリプトにだけ対応するのではなく、複数の記述に共通する構造を字句規則と文法規則に分けて整理しました。

実施内容

調査した内容をPLYのToken規則と文法規則として定義し、関数、変数宣言、式、条件分岐、ループを解析するParserを実装しました。新たに判明した記述パターンを段階的に追加できる構成とし、既存スクリプトから得た仕様をParserへ反映しました。

結果

正式な言語仕様がいない状況でも、実際に使用されているスクリプトを根拠として、変換に必要な字句・構文規則を整理し、解析処理として実装しました。

5.2 字句解析と構文解析の責務分離

課題

識別子、数値、文字列、演算子、コメントなどを正しく分類するとともに、関数や制御構文などのプログラム構造を解析する必要がありました。

判断

局所的な文字列判定には正規表現を使用し、複数のTokenからなるプログラム構造の解析はParserで行うように責務を分けました。

実施内容

- Tokenごとに正規表現を定義し、PLYのLexerで入力文字列を字句へ分割しました。
- 識別子を読み取った後に予約語一覧と照合し、**if**、**for**、**return**などを専用Tokenへ分類しました。
- 数値、16進数、複数形式の文字列、単行コメント、複数行コメントに個別の規則を実装しました。
- Lexerが分類したTokenを基に、Parserで式、宣言、関数、制御構文をASTへ変換しました。

結果

正規表現が適する字句の判定と、正規表現だけでは扱いにくい構文構造の解析を分離しました。字句の種類ごとに規則を独立させることで、調査結果に応じて規則を追加・修正できる構成を実現しました。

5.3 言語仕様の差を考慮した意味変換

課題

変換元言語とLuaでは、構文だけでなく制御構造や演算子の意味も異なるため、解析した構文をそのまま置き換えるだけでは、元のスクリプトの意味を維持できない可能性がありました。

実施内容

ASTに保持した型、スコープ、親子関係、および事前解析で収集した関数シグネチャを利用し、言語間の仕様差を補う意味変換を実装しました。構文解析と意味変換を分離することで、Parserは変換元言語の構文解析を担当し、仕様差への対応はASTを基に行う構成としました。

具体的には、以下の変換を実装しました。

- **+**演算子について、オペランドの型が文字列の場合はLua側の文字列連結処理へ変換し、数値の場合は加算として出力しました。
- C言語形式の**for**文について、初期値、比較演算子、終端値、更新量を解析し、意味を維持できる場合はLuaの数値**for**文へ変換しました。条件式をループごとに再評価する必要がある場合は、初期化処理と更新処理を含む**while**文へ変換しました。
- Luaに前置・後置の**++**および**--**演算子がないため、複合式内での評価値と変数更新の順序を保つように、必要な一時値と更新文へ展開しました。
- Luaに参照引数がないため、参照引数を持つ関数呼び出しを多値返却と多重代入へ変換し、呼び出し後の値を書き戻す処理を生成しました。

結果

単純なテキスト置換では扱いにくいプログラム構造と、言語仕様の違いを考慮してLuaコードを生成できるようにしました。

5.4 コメントを維持したLuaコード生成

課題

コードだけを変換すると、元のスクリプトに記載された説明や注意事項が失われ、変換後の保守性が低下します。そのため、コメントも対応するLuaコードへ引き継ぐ必要がありました。

実施内容

単行コメントと複数行コメントをLexerで読み取り、コメント情報として保持しました。コメントが構文要素の前後または同じ行のいずれにあるかを判定し、対応するASTノードへ関連付けました。Luaコード生成時には、保持した位置情報に応じて**--**または**--[[]]**形式で出力しました。

結果

コメントを空白として破棄せず、構文要素に付随する情報として扱うことで、変換元コードの説明や注意事項を、可能な限り対応するLuaコードの位置へ引き継げるようにしました。

6. 成果

- 対象となったスクリプトをLuaへ変換し、後続の組み込み作業で利用可能な成果物として出力しました。
- 構文解析、意味変換、コード生成を分離し、単純なテキスト置換では扱いにくいプログラム構造や言語仕様の違いを考慮した変換を実現しました。
- 元スクリプトのコメントと型注釈をLuaコードへ出力し、後続工程で継続して保守できる形にしました。

7. 本プロジェクトで得た知識・能力

- 正式な仕様がない状態から、実データを調査して言語仕様を整理する能力
- 正規表現が適する処理と、Parserによる構文解析が必要な処理を見極め、責務を分ける設計力
- Lexer、Parser、AST、意味変換、コード生成からなる言語変換処理の設計・実装能力